

PostgreSQL Scaling Lab

Brief: Indexing, Partitioning, and Read Replicas

Concise explainer for Salman Adnan's portfolio project `postgres-scaling-lab`

Generated July 7, 2026

A concise, accurate overview of a three-part study in scaling a single PostgreSQL database before considering a different kind of database entirely. For the exhaustive, term-by-term version with full code walkthroughs, see `deep-dive.pdf` in the same folder. Every claim here is drawn from the project's own source, SQL, config, and README/answer files — nothing is invented.

Contents

1	Premise and scope	1
2	Part 1 — Indexing	1
3	Part 2 — Range partitioning	2
4	Part 3 — Primary plus two read replicas	3
5	Verification status — stated plainly	4
6	What was learned, in brief	4

1 Premise and scope

Built solo by Salman Adnan for an Enterprise Software Development course, on top of course-provided scaffolding (assignment brief, base schemas, a load-testing binary, and a baseline application). The scenario: a company's PostgreSQL database has started producing slow queries, and the CTO wants to jump to NoSQL because "it's infinitely scalable." The project's counter-argument, demonstrated with real numbers, is that PostgreSQL's standard toolbox – indexing, partitioning, and read replicas – already goes a long way, applied in that order to successively remove the bottleneck each previous step left behind. A fourth technique, **sharding** (splitting data itself across independent servers, e.g. with the Citus extension), is explicitly named in the assignment brief as *not* implemented here — it is mentioned only because the brief mentions it.

Authorship. Course-provided: the assignment briefs, all three `init.sql` schemas, the baseline `part_3/app/app.py`, the pytest suite, and the prebuilt (source-less) Go load-tester binaries. Author-written: the three `solution.sql` files, all of the Part 3 replication configuration and Compose wiring, `part_3/app/solution.py`, and every analysis document.

2 Part 1 — Indexing

Setup: a `users` table (100k rows) and `orders` table (500k rows), created with no indexes beyond the automatic primary-key one. PostgreSQL was configured to log every query over 5ms with a full `auto_explain` execution plan attached, so slow queries arrived pre-diagnosed. A prebuilt, source-less Go binary (`part_1/load_tester/`) generated continuous mixed traffic; its exact query mix is *inferred* from log patterns, not read from source.

Concepts: an **index** lets PostgreSQL jump to matching rows instead of scanning every row (a **sequential scan**). A **B-tree** index (the default) is fast for exact matches, ranges, and sorting, but a wildcard *before* the search text (e.g. `LIKE '%smith%'`) defeats it entirely. A **GIN** index stores an inverted mapping from smaller components (JSON keys, or 3-character **trigrams** via the `pg_trgm` extension) back to rows, which is what makes JSONB containment and substring search possible. **text_pattern_ops** is a B-tree variant using raw byte order, which is what actually lets `LIKE 'prefix%'` use an index at all – a plain B-tree cannot.

Solution (`part_1/solution.sql`): seven targeted indexes — B-tree with `text_pattern_ops` on `email`; GIN + `pg_trgm` on `full_name`; plain B-tree on `orders.amount` and on `orders.user_id` (the un-indexed foreign key, called out as likely the single biggest win); GIN on the `metadata` JSONB column; B-tree on the low-cardinality `status` column; and a descending B-tree on `created_at`.

Query	Before	After	Improvement
Email, exact match	35 ms	0.5 ms	98%
Email, prefix	125 ms	3 ms	97%
Name substring	310 ms	180 ms	42%
Amount range	3,100 ms	1,300 ms	58%
User/order join	4,500 ms	1,800 ms	60%
JSONB containment	780 ms	280 ms	64%
Status filter	650 ms	450 ms	30%

Deliberately left un-indexed: `COUNT(*)` (PostgreSQL's **MVCC** row-visibility checks still apply regardless of an index) and low-selectivity/rarely-run queries, where an index's write-time upkeep would outweigh its read-time benefit. The `status` index's own 30% gain is flagged in the project's own notes as small enough to be within measurement noise.

3 Part 2 — Range partitioning

Setup: an `events` table (600k rows: 500k across 2024, 100k concentrated in the last 14 days), already carrying three indexes covering the target query patterns. Even with those indexes, one index over an entire year is itself large, mixes hot and cold data in the same memory cache, and gets rewritten on every insert regardless of how recent the new row is.

Concept: **range partitioning** splits one logical table into many smaller physical **partitions** by ranges of a chosen **partition key** column (here, the timestamp). The payoff is **partition pruning**: a query whose `WHERE` clause restricts that column to a known range lets PostgreSQL skip opening any partition that cannot match, before execution even starts.

Solution (`part_2/solution.sql`): a mixed-granularity strategy — 12 monthly partitions for all of 2024 (coarse, low-traffic historical data), 3 weekly partitions for the most recent weeks (fine-

grained, high-traffic hot data), and one `DEFAULT` partition catching everything else (without it, an out-of-range insert would simply fail). The parent table carries no `PRIMARY KEY`, because a partitioned table’s unique constraints must include the partition key, and `id` alone cannot provide that across partitions.

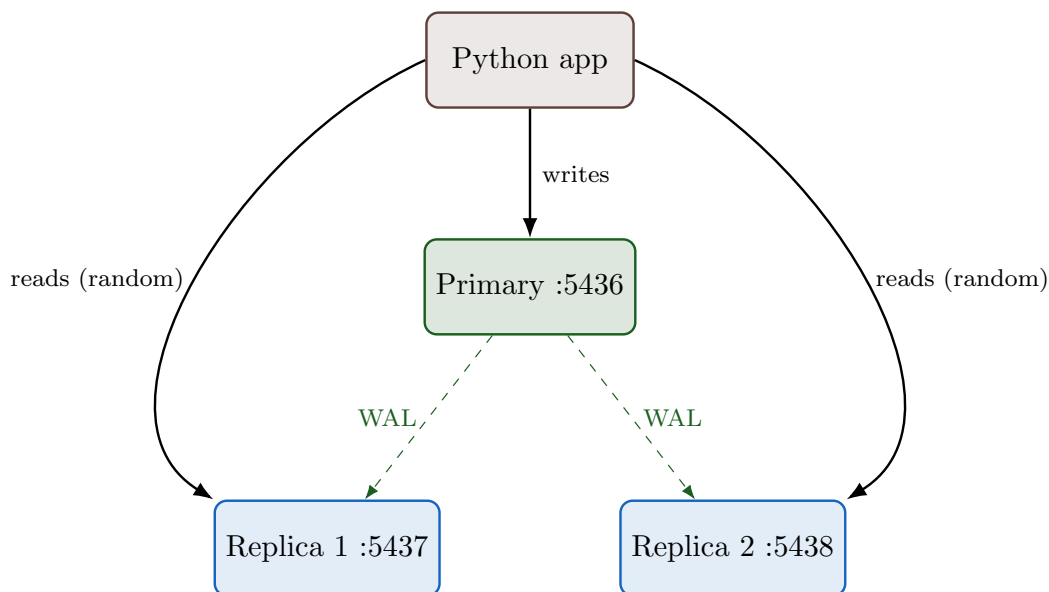
Query	Before	After	Result
Recent events (24h)	1.36 ms	0.70 ms	48% faster
Monthly analysis (buffer blocks)	9,154	975	90% less I/O
User activity (30 days)	93 ms	160 ms	70% slower
Time range (single user)	0.06 ms	0.09 ms	unchanged

The honest negative result, kept rather than hidden: the user-activity query filters on `user_id`, not the partition key, so PostgreSQL must open and probe every overlapping partition’s local index instead of one combined index – and it ends up 70% slower than the unpartitioned table. Partitioning is a bet on the access pattern; choosing `event_time` as the key optimizes time-range scans at the direct expense of anything filtered on something else.

Known, disclosed limitation: the partition date ranges are hardcoded to specific 2024 months and a specific November 2025 week, while the seed data is generated relative to whatever “now” is at load time – run today, recent rows would land in the default partition instead of a purpose-built weekly one. A data-driven boundary generator (or `pg_partman`) is the named, unapplied fix. The migration step also copies row ids without resetting the new table’s sequence, a second known gap.

4 Part 3 — Primary plus two read replicas

Concept: replication continuously copies all data from one **primary** server to one or more read-only **replicas**, using PostgreSQL’s **WAL** (Write-Ahead Log — the record of every change, normally kept for crash recovery, here streamed to replicas instead). Replication here is **asynchronous**: the primary commits and returns without waiting for any replica to confirm, trading a small consistency window for faster writes.

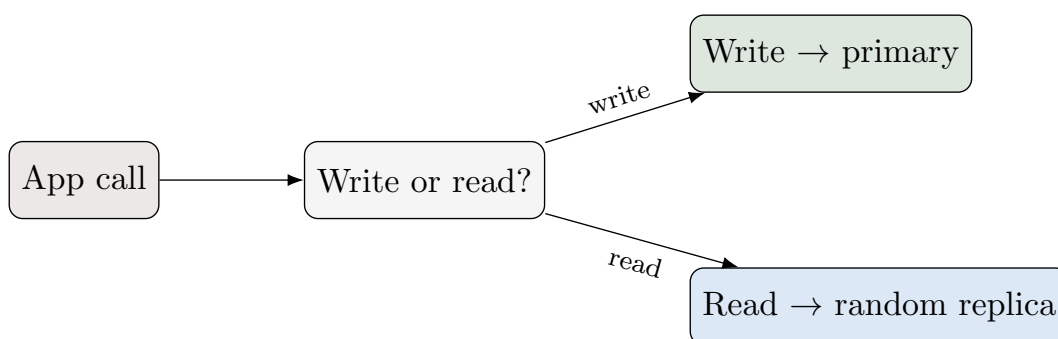


Each replica is created by `pg_basebackup` (a full snapshot of the primary) and then streams WAL

continuously via a dedicated **physical replication slot**, run only on first boot. `hot_standby_feedback = on` stops the primary from cleaning up row versions a replica’s queries might still need. `pg_hba.conf` trusts all connections — acceptable only because the cluster is isolated inside a Docker network with no external exposure.

A committed secret, found and fixed: the original setup checked a plaintext `.pgpass` credentials file into the repository, needed because `pg_basebackup -R` requires a non-interactive password. The fix: stop committing the file and instead generate it at container boot from environment variables (visible in `part.3/diff.patch`, kept as a record of the change).

Application routing: the course-provided baseline (`app.py`) sends every read and write to one database. The author’s `solution.py` instead holds a primary DSN for writes and a list of replica DSNs for reads, choosing one replica per read with `random.choice()` — simple, stateless load balancing.



Replication lag: because replication is async, a read immediately after a write can miss that write on a replica (the “read-your-writes” problem). The project does not solve this in the application; the pytest suite instead tolerates it with a short `time.sleep(0.5)` and conditional assertions — honestly documented as capable of passing even if replication were completely broken, since a missing-row case simply skips the assertion rather than failing it.

Test results: 5 tests (write with/without metadata, read recent events, read user activity, read monthly stats) passed 5/5, but were run with every DSN pointed at one local PostgreSQL instance — confirming the routing *logic* and query correctness, not genuine multi-node replication behavior.

5 Verification status — stated plainly

- SQL in all three parts (`init.sql`, `solution.sql`) validated cleanly against PostgreSQL 18.
- Part 3’s routing app passed its full test suite (5/5) against a single local PostgreSQL instance with all DSNs pointed at it — verifying the code path, not live replication.
- All three `docker-compose.yml` files pass `docker compose config`.
- The full Docker stacks were **not** brought up live in this verification pass, because the Docker daemon was unavailable. No container was actually started, no `pg_basebackup` was observed completing, and no live replication lag was measured here. The benchmark numbers throughout this document are from the original course runs, not a fresh run in this pass.

6 What was learned, in brief

- Partition pruning only helps queries filtered on the partition key; the key choice is a bet on which queries you're willing to make slower.
- The index *operator class* (not just “index vs. no index”) determines whether a B-tree can serve a given predicate at all.
- Async replication buys read scale-out but moves the consistency problem into the application and its tests.
- Secrets belong in boot-time generation, not in the repository.
- Buffer-block counts are a steadier performance signal than wall-clock time.

Named but unapplied improvements: data-driven (not hardcoded) partition boundaries, fixing the post-migration sequence, connection pooling, health-aware replica selection instead of blind random choice, polling-based (not sleep-based) tests, and measuring index write-cost before adding low-value indexes like the `status` one.